

Informe de Análisis SGJA

Sistema de Gestión de Justificaciones y Biblioteca

Fecha: 19/05/2026 **Versión:** Producción (sgja-app-blue.vercel.app)

1. Dashboards por Rol

Resumen General

Dashboard	Líneas	Queries Supabase	Responsive Móvil	Usa Componentes Compartidos
Admin	81	0 (navegación estática)	✔ Sí (CSS con clamp)	✗ No
Inspector	312	2 (listar/actualizar solicitudes)	⚠ Limitado (solo flexWrap)	✔ Card, Button
Profesor	1597	8 (bloques, cursos, estudiantes, solicitudes)	✔ Sí (detección móvil explícita)	✗ No (inline styles)
Estudiante	307	1 (historial solicitudes)	⚠ Limitado	✔ Card, Button
Apoderado	331	1 (solicitudes del pupilo)	✗ No (4 columnas fijas en móvil)	✔ Card, Button

Problemas Identificados

- **EstadoBadge** duplicado idénticamente en Inspector, Estudiante y Apoderado → extraer a `components/Common/`
- Admin y Profesor no usan componentes compartidos (Button, Card) a diferencia del resto
- Ningún dashboard usa lazy loading o Suspense
- Dashboard Apoderado: 4 columnas de estadísticas fijas se ven mal en pantallas pequeñas

Recomendaciones

1. Estandarizar todos los dashboards para usar componentes Common
2. Agregar detección de móvil (`innerWidth < 768`) en Inspector, Estudiante y Apoderado
3. Extraer EstadoBadge a componente compartido
4. Dashboard Apoderado: cambiar grid a 2 columnas en móvil

2. Mantenedores (CRUD)

Estado por Archivo

Archivo	Líneas	Móvil?	Paginación?	Usa Common?	Estado
GestionUsuarios	1196	✔ Sí	✔ Sí	✔ Sí	✔ Completo

Archivo	Líneas	Móvil?	Paginación?	Usa Common?	Estado
MantenedorLibros	254	✗ No	✓ Sí	✓ Button/Modal	✓ Completo
MantenedorEstudiantes	1282	✗ No	✓ Sí	✗ No	✓ Completo
Inventario	185	✗ No	✓ Sí	✗ No	✓ Completo
HistorialBiblioteca	110	✗ No	✓ Sí	✗ No	✓ Completo
SolicitudesRegistro	328	✗ No	✓ Server-side	✗ No	✓ Completo
MantenedorCursos	596	✗ No	✗ No paginación	✗ No	⚠ Riesgo >500 registros
MantenedorFuncionarios	778	✗ No	✗ No paginación	✗ No	✗ Incompleto (TODOs)
MantenedorMotivos	616	✗ No	✗ No paginación	✗ No	⚠ Riesgo >500 registros
MantenedorRolesPage	23	—	—	—	✓ Wrapper

Riesgos

- **MantenedorCursos, MantenedorFuncionarios, MantenedorMotivos:** sin paginación, cargan todos los registros. Con más de 500 registros el rendimiento se degrada significativamente
- **MantenedorFuncionarios:** tiene servicios comentados como TODO, funcionalidad incompleta

Recomendaciones

1. **Prioridad alta:** Agregar paginación a Cursos, Funcionarios y Motivos
2. **Prioridad alta:** Completar la implementación de MantenedorFuncionarios
3. **Prioridad media:** Agregar vista móvil (tabla → tarjetas) en MantenedorEstudiantes y MantenedorLibros

3. Módulos Funcionales

Biblioteca

Ruta	Componente	Roles
/libros	MantenedorLibros	ADMIN
/catalogo	Catalogo	ADMIN, ESTUDIANTE
/prestamos	Circulacion	ADMIN
/inventario	Inventario	ADMIN
/historial-biblioteca	HistorialBiblioteca	ADMIN
/config-biblioteca	ConfigBiblioteca	ADMIN

Estado:  Completo. Soportar: préstamo, devolución, renovación, email, multas, suspensiones.

Secretaría

Ruta	Componente	Roles
/secretaria	— (menú padre)	ADMIN
/mantenedor-funcionarios	MantenedorFuncionarios	ADMIN

Estado:  Incompleto. MantenedorFuncionarios tiene backend sin implementar (TODOs).

Justificaciones

Ruta	Componente	Roles (Sidebar)	Roles (AppContent)
/registrar	RegistrarJustificacion	ADMIN, INSPECTOR	ADMIN, INSPECTOR, PROFESOR
/ver-justificaciones	VerJustificaciones	ADMIN, INSPECTOR	ADMIN, INSPECTOR
/gestion-pases	GestionPases	ADMIN, INSPECTOR, PROFESOR	ADMIN, INSPECTOR, PROFESOR
/justificaciones	JustificacionesAtrasos	—	ADMIN, INSPECTOR

Inconsistencia: PROFESOR tiene acceso a **/registrar** en AppContent pero el sidebar no lo muestra.

Préstamos (Circulacion.tsx)

Operaciones soportadas:

-  **Prestar:** búsqueda de libro, selección de estudiante, slider de días, validación de reglas
-  **Devolver:** condición (bueno/dañado/perdido), multa, justificación, suspensión
-  **Renovar:** modal +2/+4/+7 días, límite de renovaciones
-  **Email:** selección múltiple, envío de avisos de vencimiento
-  **Filtros:** por estado, rango de fecha, paginación

4. Menú por Rol

Sidebar (Layout.tsx)

Rol	Items Visibles
ADMIN	Inicio, Secretaría > Funcionarios, Justificaciones (3), Biblioteca (6), Monitoreo (2), Seguridad, Configuración (11)
INSPECTOR	Inicio, Justificaciones (3), Panel QR, Seguridad
PROFESOR	Inicio, Justificaciones > G. Pases, Mis estudiantes, Seguridad, Configuración > G. Usuarios

Rol	Items Visibles
ESTUDIANTE	Inicio, Biblioteca > Catálogo, Mi historial, Seguridad
APODERADO	Inicio, Mi pupilo, Seguridad

Mobile Footer (MobileLayout.tsx)

Rol	Botones
ADMIN	Inicio, Catálogo, Registrar, Config (4)
INSPECTOR	Inicio, Registrar, QR (3)
PROFESOR	Solo Inicio (1)
ESTUDIANTE	Inicio, Catálogo (2)
APODERADO	Solo Inicio (1)

Problemas

- PROFESOR y APODERADO tienen solo 1 botón en móvil → muy limitado
- PROFESOR debería tener acceso rápido a "Mis estudiantes" y "G. Pases" en mobile
- APODERADO debería tener acceso rápido a "Mi pupilo"

5. Acceso Estudiante y Apoderado

ESTUDIANTE

Funcionalidad	Estado
Dashboard con historial de justificaciones	✓
Catálogo de biblioteca (solo lectura)	✓
Mi historial personal	✓
Seguridad (cambio de contraseña)	✓
Editar datos personales	✗ No disponible
Notificaciones	✗ No implementado
Estado bibliotecario (multas/suspensiones)	✗ No visible
Solicitar préstamo	✗ No (solo ADMIN presta)

APODERADO

Funcionalidad	Estado
Dashboard con justificaciones del pupilo	✓

Funcionalidad	Estado
Mi pupilo (información)	✓
Seguridad	✓
Editar datos personales	✗ No disponible
Notificaciones del pupilo	✗ No implementado
Estado bibliotecario del pupilo	✗ No visible

6. Seguridad — 2FA / Autenticación

Estado Actual

- **Proveedor único:** Google OAuth
- **Sin 2FA/MFA:** no hay TOTP, SMS, ni authenticator app
- **Sin RLS:** no hay políticas de seguridad a nivel de base de datos
- **Clave anónima:** el cliente usa `SUPABASE_ANON_KEY` — sin RLS, cualquier request autenticado puede leer todas las tablas
- **Seguridad en código:** las validaciones de rol se hacen en `useAuth` (frontend), no en la BD

Riesgos

- Si alguien obtiene el `access_token` de Supabase (ej: XSS, localStorage expuesto), puede leer cualquier tabla
- No hay rate limiting en login (solo el que provee Google)
- No hay log de intentos de acceso fallidos

Recomendaciones

1. Implementar RLS en tablas críticas (usuarios, datospersonalesusuarios)
2. Agregar log de accesos (quién, cuándo, IP)
3. Evaluar 2FA con Supabase Auth MFA (soporta TOTP)
4. Rotar la `service_role key` si está expuesta en el código

7. Auto-logout por Inactividad

Estado: ✓ Implementado

Parámetro	Valor
Tiempo de inactividad	10 minutos (configurable vía Supabase)
Advertencia previa	60 segundos (countdown modal)
Eventos que reinician	mousedown, keydown, scroll, touchstart, click
Extensión de sesión	Botón "Continuar sesión"
Cierre automático	sí, al llegar a 0

8. Consumo de Aplicación

Estado Actual

- **Sin monitoreo:** no hay Vercel Analytics, Sentry, PostHog, ni contador de queries
- **Sin logs de rendimiento:** no se mide tiempo de respuesta ni cantidad de requests
- **Cache existente:** `cacheService.ts` usa IndexedDB, pero no todas las consultas lo usan
- **PWA:** Service Worker con StaleWhileRevalidate solo para Supabase REST API

Posibles Cuellos de Botella

1. Mantenedores sin paginación (Cursos, Funcionarios, Motivos) → queries sin límite
2. Dashboard Profesor hace 8 queries simultáneas al cargar
3. Tablas sin índices en `library_loans.student_id` y `book_copies.book_id`
4. El login sin RLS significa que cualquier consulta pasa por el anon key sin filtros de BD

Recomendaciones

1. Agregar Vercel Analytics (gratis, 1 clic en dashboard)
 2. Agregar índices a las tablas más consultadas
 3. Implementar paginación server-side en lugar de client-side para conjuntos grandes
 4. Medir antes/después de cada cambio para evaluar impacto
-

9. Simulación 10+ Usuarios Concurrentes

Estimación de Carga por Usuario Activo

Acción	Queries Aprox
Login (Google OAuth + useAuth)	3-4 (auth + usuarios + datos personales)
Dashboard (según rol)	0-8 queries
Navegación entre páginas	1-3 queries por página
Préstamo de libro	4-6 queries (validaciones + insert + update)
Devolución	3-5 queries
CRUD en mantenedor	1-2 queries

Escenario: 10 usuarios simultáneos

- ~30-80 requests/minuto en horario punta
- Supabase Free Plan: 50,000 rows/month, 2GB bandwidth — suficiente para este volumen
- Vercel Free Plan: 100,000 serverless invocations/month — suficiente

Riesgos

- Los mantenedores sin paginación multiplican el row count en Supabase

- Si cada profesor consulta "todos los estudiantes" sin paginación, 20 profesores × 500 estudiantes = 10,000 rows en una sola consulta
- Supabase Free Plan limita a 50,000 rows totales — esto se puede alcanzar fácilmente

Recomendaciones

1. Agregar paginación server-side obligatoria en todas las consultas de listados
2. Monitorear el contador de rows en Supabase Dashboard
3. Considerar migrar a Supabase Pro (\$25/mes) si se superan los límites

10. SQLite + Sincronización en la Nube

Análisis de Viabilidad

No recomendado para la arquitectura actual.

Aspecto	Evaluación
Auth	✗ Depende de Supabase (no hay SQLite auth)
RLS	✗ Sin RLS en BD, SQLite no tendría seguridad
Joins	✗ Biblioteca usa joins entre 3-4 tablas constantemente
Complejidad	✗ Sync engine offline/online es complejo de implementar y mantener
Beneficio real	⚠ Bajo — la app ya funciona online con buena latencia (us-west-1)

Alternativa Recomendada

Seguir usando `cacheService.ts` (IndexedDB) que ya existe:

- Cachea respuestas de Supabase
- TTL configurable
- No requiere sync engine (solo invalidación por tiempo)
- Evita roundtrips innecesarios sin la complejidad de SQLite

Solo valdría la pena SQLite si se requiere modo offline total (sin internet).

11. Módulo de Temas / Personalización

Estado Actual

- Dark/light mode básico vía `useTheme.ts`
- Booleano en localStorage (`sgja_tema_oscuro`)
- Aplica `data-theme="dark"` / `data-theme="light"` en `<html>`
- Toggle sol/luna en Header.tsx y Parametros.tsx
- Colores fijos en `COLORES_TEMA` (claro/oscuro)

Para Hacerlo Extensible

1. Cambiar de booleano a string: `'claro' | 'oscuro' | 'personalizado'`
 2. Guardar paleta de colores en Supabase (`configuracion_temas`)
 3. Editor de temas: color primario, secundario, acento, fondo, texto
 4. Guardar preferencia por usuario (no global)
 5. Aplicar variables CSS en runtime mediante CSS custom properties
-

12. Funcionalidades Faltantes

Pendientes de tu Lista Original

- ☐ **Dashboard por rol** — revisar y estandarizar responsive
- ☐ **Mantenedores** — paginación y vista móvil
- ☐ **Secretaría** — completar MantenedorFuncionarios
- ☐ **Justificaciones** — corregir inconsistencia PROFESOR
- ☐ **Préstamos** — probar flujo completo
- ☐ **Menú navbar** — PROFESOR y APODERADO tienen solo 1 botón en móvil
- ☐ **Acceso estudiante/apoderado** — falta editar perfil, notificaciones, estado bibliotecario
- ☐ **2FA** — evaluar implementación
- ☐ **Auto-logout** — ya implementado
- ☐ **Consumo** — agregar monitoreo
- ☐ **Simulación 10+ usuarios** — paginación server-side
- ☐ **SQLite** — no recomendado por ahora
- ☐ **Temas** — hacer extensibles

Adicionales Detectados en el Análisis

Mantenedor de Ubicaciones (Estantería/Fila)

- `book_copies.estanteria` y `book_copies.fila` son **texto libre**
- No hay tabla maestra de ubicaciones
- Los usuarios escriben valores inconsistentes ("1", "01", "Estante 1")
- **Solución:** crear tabla `ubicaciones` (id, estanteria, fila, columna, id_establecimiento) con CRUD, y usar `<select>` en Inventario

Mantenedor de Categorías

- `books.categoria` es **texto libre**
- Mismo problema: "Novela", "novela", "NOVELA" aparecen como distintas
- **Solución:** tabla `categorias` con CRUD y select en formularios de libro

Reportes y Estadísticas

- Ruta `/reportes` existe pero hay que verificar si tiene datos reales
- Sugerencia: préstamos por mes, libros más prestados, top estudiantes, multas recaudadas

Carga Masiva de Estudiantes (CSV)

- Ya existe para libros (MantenedorLibros)

- Crear equivalente para estudiantes: importar desde CSV con validación de RUT

Log de Accesos

- Registrar inicio/cierre de sesión: quién, cuándo, IP, user-agent
 - Diferente del monitoreo de operaciones (que ya registra CRUDs)
 - Util para auditoría de seguridad
-

Prioridades Sugeridas

● Crítico (Corto Plazo)

1. Paginación en MantenedorCursos, MantenedorFuncionarios, MantenedorMotivos
2. Completar MantenedorFuncionarios (backen y frontend)
3. Corregir inconsistencia de rutas PROFESOR (sidebar vs AppContent)

● Importante (Mediano Plazo)

4. Responsive móvil en dashboards Inspector, Estudiante, Apoderado
5. Ubicaciones estructuradas (estantería/fila) en Biblioteca
6. Categorías de libros estructuradas
7. Vista móvil en mantenedores principales

● Mejora (Largo Plazo)

8. Módulo de temas extensibles
9. Reportes y estadísticas
10. Carga masiva de estudiantes CSV
11. Log de accesos

● Opcional

12. 2FA / MFA
 13. Modo offline con SQLite (solo si es necesario)
-

Fin del informe.